

Base de Datos - AutoVentas Pro

Tecnologia

- **Base de datos:** PostgreSQL 15 hosteada en Supabase (aws-1-us-east-2)
 - **ORM:** Prisma 7 con adapter [@prisma/adapters-pg](#)
 - **Pool de conexiones:** [pg.Pool](#) con session mode (puerto 5432)
 - **Almacenamiento imagenes:** Supabase Storage (bucket [vehiculos](#))
-

Configuracion

Variables de entorno (.env)

```
# PostgreSQL directo via pooler Supabase
DATABASE_URL=postgresql://postgres.ickuktmrjugshjzagfmu:password@aws-1-us-east-2.pooler.supabase.com:5432/postgres
DB_HOST=aws-1-us-east-2.pooler.supabase.com
DB_PORT=5432
DB_NAME=postgres
DB_USER=postgres.ickuktmrjugshjzagfmu
DB_PASSWORD=password

# Supabase Storage
SUPABASE_URL=https://ickuktmrjugshjzagfmu.supabase.co
SUPABASE_ANON_KEY=eyJ...anon
SUPABASE_BUCKET=vehiculos
```

Archivos de configuracion

Archivo	Proposito
prisma/schema.prisma	Esquema de datos (modelos Prisma)
prisma.config.ts	Configuracion Prisma v7 (schema path, datasource URL)
prisma/seed.ts	Semilla de datos iniciales

Conexion a la DB (runtime)

[src/infrastructure/database/prisma.service.ts](#) — Singleton que crea un pool [pg.Pool](#) + [PrismaPg](#) adapter y exporta [getPrisma\(\)](#).

```
const pool = new pg.Pool({
  host: process.env['DB_HOST'],
  port: Number(process.env['DB_PORT']),
  database: process.env['DB_NAME'],
  user: process.env['DB_USER'],
```

```
password: process.env['DB_PASSWORD'],
});
const adapter = new PrismaPg(pool);
prisma = new PrismaClient({ adapter });
```

Modelos (Prisma Schema)

Empleado (**empleado**)

Nominas del personal.

Columna	Tipo	Notas
id	String (UUID)	Primary key
cedula	String	Cedula identidad
nombres	String	Nombre completo
salario	Float	Salario mensual
departamento	String	Departamento
activo	Boolean	Default true
createdAt	DateTime	Auto
updatedAt	DateTime	Auto

Usuario (**usuario**)

Credenciales de acceso al sistema con roles.

Columna	Tipo	Notas
id	String (UUID)	Primary key
usuario	String	Unique, nombre de login
passwordHash	String	Hash bcrypt
rol	String	administrador, jefe_ventas, asesor
activo	Boolean	Default true
ultimoAcceso	DateTime?	Nullable
especialidad	String?	Solo asesor
experienciaAnios	Int?	Solo asesor
metaMensual	Float?	Solo asesor
areaResponsable	String?	Solo jefe_ventas
bonoGestion	Float?	Solo jefe_ventas

Columna	Tipo	Notas
nivelPermiso	String?	Solo administrador
fechaCreacion	DateTime	Auto

Vehiculo (vehiculo)

Catalogo de vehiculos disponibles.

Columna	Tipo	Notas
id	String (UUID)	Primary key
marca	String	
modelo	String	
anio	Int	
precioBase	Float	Precio sin descuentos
tipo	String	SUV, SEDAN, HATCHBACK, PICKUP, DEPORTIVO, ELECTRICO, HIBRIDO
stock	Int	Default 0
especificaciones	String?	JSON con motor, transmision, etc.
imagen	String?	URL publica desde Supabase Storage
activo	Boolean	Default true
createdAt	DateTime	Auto
updatedAt	DateTime	Auto

PlanFinanciamiento (plan_financiamiento)

Planes de financiamiento.

Columna	Tipo	Notas
id	String (UUID)	Primary key
nombre	String	Plan Basico, Estandar, Premium, Rapido
entradaMinima	Float	Porcentaje minimo de entrada
tasalInteresAnual	Float	Tasa de interes anual
plazosDisponibles	String	JSON array de meses (ej: [12,24,36])
comision	Float	Default 0
activo	Boolean	Default true
createdAt	DateTime	Auto
updatedAt	DateTime	Auto

Cotizacion (**cotizacion**)

Cotizaciones generadas por clientes.

Columna	Tipo	Notas
id	String (UUID)	Primary key
vehiculoid	String	FK a Vehiculo
planId	String	FK a PlanFinanciamiento
clienteNombre	String	
clienteEmail	String	
clienteTelefono	String	
clienteCedula	String?	Nullable (futuras migraciones haran required)
clienteCiudad	String?	Nullable
precioVehiculo	Float	
entrada	Float	
montoFinanciado	Float	precioVehiculo - entrada
plazoMeses	Int	
tasalInteresAnual	Float	
cuotaMensual	Float	Calculada por sistema frances
tablaAmortizacion	String	JSON array de objetos cuota
estado	String	Default PENDIENTE . Valores: PENDIENTE, APROBADA, RECHAZADA, EXPIRADA
createdAt	DateTime	Auto
updatedAt	DateTime	Auto

Venta (**venta**)

Venta asociada a una cotizacion aprobada.

Columna	Tipo	Notas
id	String (UUID)	Primary key
cotizacionId	String	Unique FK a Cotizacion
asesorId	String	FK a Usuario (rol asesor)
valorTotal	Float	
estado	String	Default PENDIENTE

Columna	Tipo	Notas
createdAt	DateTime	Auto
updatedAt	DateTime	Auto

Migraciones aplicadas

#	Fecha	Nombre	Cambios
1	2026-07-06	init	Creacion de tablas Empleado, Usuario, Vehiculo, PlanFinanciamiento
2	2026-07-08	update_vehiculo_plan	Agregado campo especificaciones a Vehiculo, comision a Plan
3	2026-07-08	add_cotizacion_venta	Creacion de tablas Cotizacion, Venta
4	2026-07-09	add_vehiculo_imagen	Agregado campo imagen a Vehiculo
5	2026-07-09	add_cliente_cedula_ciudad	Agregados campos clienteCedula, clienteCiudad a Cotizacion

Comandos utiles:

```
npm run prisma:migrate      # Crear nueva migracion
npx prisma migrate dev      # Aplicar migraciones
npx prisma studio           # Abrir UI de exploracion de datos
```

Repositorios

Arquitectura Hexagonal

Cada modulo sigue el patron **Domain** → **Infrastructure**:

```
src/domain/repositories/    ← Interfaces (contratos)
src/infrastructure/repositories/ ← Implementaciones (Prisma/Supabase)
src/interfaces/routes/      ← Wire de dependencias
```

Repositorios activos

Interface	Implementacion	Usado en
empleado.repository.ts	empleado.supabase.repository.ts	empleadoRoutes.ts

Interface	Implementacion	Usado en
<code>usuario.repository.ts</code>	<code>usuario.supabase.repository.ts</code>	<code>authRoutes.ts</code> , <code>usuarioRoutes.ts</code>
<code>IVehiculoRepository.ts</code>	<code>vehiculo.supabase.repository.ts</code>	<code>vehiculoRoutes.ts</code> , <code>cotizacionRoutes.ts</code>
<code>IPlanFinanciamientoRepository.ts</code>	<code>plan.supabase.repository.ts</code>	<code>planRoutes.ts</code> , <code>cotizacionRoutes.ts</code>
<code>ICotizacionRepository.ts</code>	<code>cotizacion.supabase.repository.ts</code>	<code>cotizacionRoutes.ts</code> , <code>ventaRoutes.ts</code>
<code>IVentaRepository.ts</code>	<code>venta.supabase.repository.ts</code>	<code>ventaRoutes.ts</code>

No usado

Interface	Implementacion	Motivo
<code>IAsesorRepository.ts</code>	<code>AsesorRepositoryImpl.ts</code>	Modulo Asesor nunca se conecto en <code>app.ts</code>

Patron de implementacion

Cada repositorio:

1. Obtiene instancia de Prisma via `getPrisma()`
2. Convierte entre `PrismaModel` y entidad del dominio via `Entity.desdeDatos()`
3. Usa `entity.toJSON()` para persistir

Ejemplo (`empleado.supabase.repository.ts`):

```
async findById(id: string): Promise<Empleado | null> {
  const prisma = getPrisma();
  const data = await prisma.empleado.findUnique({ where: { id } });
  return data ? Empleado.desdeDatos(data) : null;
}
```

Supabase Storage

Bucket `vehiculos`

- Tipo: **publico** (los archivos son accesibles via URL publica)
- Proposito: Almacenar imagenes de vehiculos

RLS Policies

Se configuraron 3 policies para el rol `anon`:

Policy	Operacion	Descripcion
INSERT	INSERT	Permite subir archivos desde el frontend
SELECT	SELECT	Permite leer/listar archivos
DELETE	DELETE	Permite eliminar archivos

Como se usa

src/infrastructure/storage/supabase.storage.ts — Cliente Supabase que:

1. Verifica que el bucket existe (lo crea si no)
2. Sube archivos con `supabase.storage.from('vehiculos').upload()`
3. Retorna URL publica via `getPublicUrl()`

Flujo de upload (VehiculoForm):

```
Frontend (input file)
→ POST /api/upload (UploadController)
→ uploadFile() (supabase.storage.ts)
  → Supabase Storage
    → retorna URL publica
→ Se guarda URL en campo `imagen` del Vehiculo
```

Seed

prisma/seed.ts — Poblacion inicial de datos.

Ejecutar con:

```
npm run seed
```

Datos sembrados

Usuarios (4):

Usuario	Password	Rol
admin	admin123	administrador
jventas	jefe123	jefe_ventas
carlos	asesor123	asesor
maria	asesor123	asesor

Empleados (5): Ana Garcia, Carlos Lopez, Maria Perez, Pedro Ramirez, Laura Jimenez.

Vehiculos (8): Toyota Corolla, Honda CR-V, Chevrolet Sail, Toyota Hilux, Ford Mustang, Nissan Leaf, Hyundai Elantra, Kia Sportage.

Planes (4): Basico (20% entrada, 8.5%), Estandar (15%, 6.5%), Premium (10%, 4.5%), Rapido (30%, 5.0%).

Cotizacion (1): Ejemplo con amortizacion a 24 meses.

Venta (1): Vinculada a la cotizacion de ejemplo, asignada al primer asesor.

Comandos utiles

```
# Desarrollo
npm run dev           # Iniciar servidor backend
npm run seed          # Sembrar datos iniciales
npm run prisma:generate  # Regenerar Prisma Client
npx prisma studio      # Explorar DB via navegador

# Migraciones
npx prisma migrate dev      # Aplicar + crear migracion
npx prisma migrate dev --name descripcion  # Crear migracion con nombre
npx prisma migrate dev --create-only       # Solo crear (sin aplicar)

# Tests
npm run test              # Todos los tests
npm run test:cotizaciones # Tests de cotizacion
npm run test:all          # Suite completa
```